PRODUCTION ARCHITECTURE & DEPLOYMENT SPECIFICATION

Hardened SecXNG: Dedicated Cybersecurity & OSINT Search Appliance

Production-Ready Ubuntu Server Deployment Guide with Tor Proxy Routing, Valkey Caching, Caddy TLS/Port 80 Reverse Proxy, and Custom Vulnerability Engines.

Target OS	Ubuntu Server 24.04 LTS (Noble Numbat)	**Stack**	Docker Compose + SearXNG + Valkey + Alpine Tor + Caddy
Architecture	Dedicated Server / VM / Mini PC	**Security Posture**	UFW Host-Locked, Loopback Bound, Hybrid Tor/Direct Routing

1. Executive Summary & Architecture Overview

SecXNG is a hardened, self-hosted metasearch appliance optimized for cybersecurity research, vulnerability intelligence, and OSINT reconnaissance. Running on a dedicated Ubuntu Server instance via Docker Compose, this architecture combines:

- **SearXNG Core:** Uncapped JSON/CSV API outputs for local LLM integration and automation, with safe search disabled and autocomplete stripped to prevent telemetry leakage.
- **Alpine Tor Sidecar:** A modern, stable Tor SOCKS5h daemon routing general web reconnaissance over onion circuits to prevent rate-limiting and track exposure.
- **Hybrid Engine Routing:** Direct connection bypass rules for high-volume or strict APIs (like `crt.sh` and NIST NVD) to eliminate Tor exit-node timeouts.
- **Valkey Caching Layer:** High-performance Redis-compatible in-memory caching for low-latency query handling.
- **Caddy Reverse Proxy:** Seamless HTTP/HTTPS front-door termination routing requests securely into the container stack.

2. Ubuntu Server Provisioning & Firewall Hardening

Log in to your fresh Ubuntu Server 24.04 LTS installation via SSH and prepare the host environment:

```
ssh youruser@<YOUR-SERVER-IP>
sudo apt update && sudo apt full-upgrade -y
sudo apt install -y curl git ca-certificates gnupg lsb-release ufw fail2ban jq openssl
```

Configuring UFW & Understanding the Docker Port Binding Trap

> **CRITICAL SECURITY NOTE: Docker & UFW Interaction**
>
> When publishing ports in Docker (e.g., `ports: - "8080:8080"`), Docker manipulates iptables/nftables PREROUTING rules directly, bypassing UFW. To prevent unauthorized exposure, SearXNG is bound strictly to `127.0.0.1` (loopback) inside Docker, forcing all external LAN access through Caddy on standard ports (80/443).

```
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow SSH, HTTP, and HTTPS strictly from your local LAN subnet (adjust to your subnet)
sudo ufw allow from 192.168.0.0/16 to any port 22 proto tcp comment 'LAN SSH'
sudo ufw allow from 192.168.0.0/16 to any port 80 proto tcp comment 'LAN HTTP'
sudo ufw allow from 192.168.0.0/16 to any port 443 proto tcp comment 'LAN HTTPS'
```

```
sudo ufw --force enable
sudo ufw status verbose
```

3. Installing Official Docker Engine & Compose v2

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc

echo   "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://
download.docker.com/linux/ubuntu   $(. /etc/os-release && echo "$VERSION_CODENAME") stable" |   sudo tee /
etc/apt/sources.list.d/docker.list > /dev/null

sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER
newgrp docker
```

4. Directory Setup & Deployment Manifests

Create the self-contained deployment directory at `/opt/secxng`:

```
sudo mkdir -p /opt/secxng/searxng /opt/secxng/caddy_data /opt/secxng/caddy_config /opt/secxng/valkey_data
sudo chown -R $USER:$USER /opt/secxng
cd /opt/secxng
```

4.1 Docker Compose Stack (`docker-compose.yml`)

```
cat << 'EOF' > /opt/secxng/docker-compose.yml
services:
  valkey:
    image: docker.io/valkey/valkey:8-alpine
    container_name: secxng-valkey
    restart: unless-stopped
    command: valkey-server --save 30 1 --loglevel warning
    volumes:
      - ./valkey_data:/data
    networks:
      secxng_net:
        ipv4_address: 172.30.0.11
    cap_drop:
      - ALL
    cap_add:
      - SETGID
      - SETUID
      - DAC_OVERRIDE

  tor-proxy:
    image: docker.io/library/alpine:latest
    container_name: secxng-tor
    restart: unless-stopped
    command: >
      sh -c "apk add --no-cache tor && echo 'SocksPort 0.0.0.0:9050' > /etc/tor/torrc && exec tor -f /etc/
tor/torrc"
    networks:
      secxng_net:
        ipv4_address: 172.30.0.10

  searxng:
    image: docker.io/searxng/searxng:latest
    container_name: secxng-core
    restart: unless-stopped
    depends_on:
      - valkey
      - tor-proxy
```

```
    ports:
      - "127.0.0.1:8080:8080"
    volumes:
      - ./searxng:/etc/searxng:rw
    environment:
      - SEARXNG_BASE_URL=http://localhost:8080/
      - UWSGI_WORKERS=2
      - UWSGI_THREADS=2
    networks:
      secxng_net:
        ipv4_address: 172.30.0.12
    cap_drop:
      - ALL
    cap_add:
      - CHOWN
      - SETGID
      - SETUID

  caddy:
    image: docker.io/library/caddy:2-alpine
    container_name: secxng-caddy
    restart: unless-stopped
    depends_on:
      - searxng
    ports:
      - "80:80"
      - "443:443"
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - ./caddy_data:/data
      - ./caddy_config:/config
    networks:
      secxng_net:
        ipv4_address: 172.30.0.13

networks:
  secxng_net:
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.0.0/24
EOF
```

4.2 Caddy Reverse Proxy Configuration (`Caddyfile`)

```
cat << 'EOF' > /opt/secxng/Caddyfile
:80 {
    reverse_proxy 172.30.0.12:8080 {
        header_up X-Forwarded-For {remote_host}
        header_up X-Real-IP {remote_host}
    }
}
EOF
```

4.3 Hardened SearXNG Configuration (`searxng/settings.yml`)

```
cat << 'EOF' > /opt/secxng/searxng/settings.yml
use_default_settings: true

server:
  bind_address: "0.0.0.0"
  port: 8080
  secret_key: "REPLACE_WITH_SECRET"
  limiter: false
  image_proxy: true
  method: "POST"

search:
  safe_search: 0
```

```
  autocomplete: ""
  formats:
    - html
    - json
    - csv
    - rss

outgoing:
  request_timeout: 15.0
  max_request_timeout: 30.0
  useragent_suffix: ""
  proxies:
    all://:
      - socks5h://172.30.0.10:9050
  using_tor_proxy: true

valkey:
  url: valkey://172.30.0.11:6379/0

engines:
  - name: nvd
    engine: xpath
    shortcut: cve
    categories: [security]
    search_url: "https://nvd.nist.gov/vuln/search/results?
form_type=Basic&results_type=overview&query={query}"
    url_xpath: '//table[@data-testid="vuln-results-table"]//a[starts-with(@data-testid, "vuln-detail-
link-")]/@href'
    title_xpath: '//table[@data-testid="vuln-results-table"]//a[starts-with(@data-testid, "vuln-detail-
link-")]'
    content_xpath: '//table[@data-testid="vuln-results-table"]//p[starts-with(@data-testid, "vuln-
summary-")]'
    disabled: false
    timeout: 15.0
    proxy: false

  - name: crtsh
    engine: json_engine
    shortcut: crt
    categories: [osint, security]
    search_url: "https://crt.sh/?q={query}&output=json"
    url_prefix: "https://crt.sh/?id="
    results_query: ""
    url_query: "id"
    title_query: "common_name"
    content_query: "name_value"
    disabled: false
    timeout: 15.0
    proxy: false

  - name: exploit-db
    engine: github
    shortcut: edb
    categories: [security]
    search_url: "https://github.com/offensive-security/exploitdb/search?q={query}"
    disabled: false
    timeout: 12.0

  - name: gtfobins
    engine: duckduckgo
    shortcut: gtfo
    categories: [security, it]
    search_url: "https://html.duckduckgo.com/html/?q=site:gtfobins.github.io+{query}"
    disabled: false

  - name: hacktricks
    engine: duckduckgo
    shortcut: ht
    categories: [security]
    search_url: "https://html.duckduckgo.com/html/?q=site:book.hacktricks.xyz+{query}"
```

```
    disabled: false
EOF
```

5. Initialization, Permissions & Launch

Inject a secure 32-byte cryptographic secret key, correct container volume ownership for UID `977`, and launch the stack:

```
sed -i "s/REPLACE_WITH_SECRET/$(openssl rand -hex 32)/g" /opt/secxng/searxng/settings.yml
sudo chown -R 977:977 /opt/secxng/searxng
docker compose down && docker compose up -d
```

> **Verification:** Run `docker compose ps`. All four containers (`secxng-core` , `secxng-valkey` , `secxng-tor` , `secxng-caddy`) should report status `Up` .

6. Custom Bangs Reference Table

Shortcut	Engine Name	Category	Description / Query Example
`!cve`	NIST NVD	`security`	`!cve openssh` — NIST vulnerability search (Direct route)
`!crt`	crt.sh OSINT	`osint`	`!crt ubuntu.com` — Certificate transparency subdomain enum
`!edb`	Exploit-DB	`security`	`!edb vsftpd` — GitHub-indexed exploit payload search
`!gtfo`	GTFOBins	`security`	`!gtfo find` — Unix binary privilege escalation methods
`!ht`	HackTricks	`security`	`!ht kerberoasting` — Pentesting methodology cheat sheets